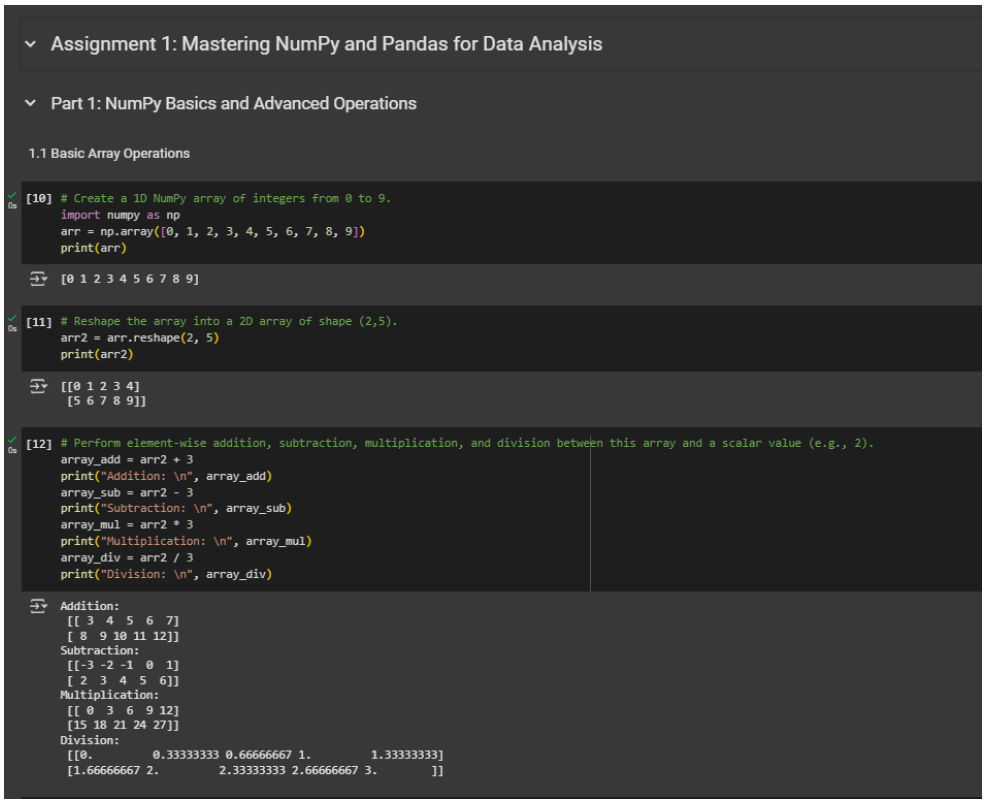
# *Assignment 1: Mastering NumPy and Pandas for Data Analysis*

## *Maram Alhusami S21207443 – sec 1*

## *Part 1 Outputs:*



Assignment 1: Mastering NumPy and Pandas for Data Analysis

Part 1: NumPy Basics and Advanced Operations

1.1 Basic Array Operations

```
[10] # Create a 1D NumPy array of integers from 0 to 9.
     import numpy as np
     arr = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
     print(arr)
```

```
[0 1 2 3 4 5 6 7 8 9]
```

```
[11] # Reshape the array into a 2D array of shape (2,5).
     arr2 = arr.reshape(2, 5)
     print(arr2)
```

```
[[0 1 2 3 4]
 [5 6 7 8 9]]
```

```
[12] # Perform element-wise addition, subtraction, multiplication, and division between this array and a scalar value (e.g., 2).
     array_add = arr2 + 3
     print("Addition: \n", array_add)
     array_sub = arr2 - 3
     print("Subtraction: \n", array_sub)
     array_mul = arr2 * 3
     print("Multiplication: \n", array_mul)
     array_div = arr2 / 3
     print("Division: \n", array_div)
```

```
Addition:
 [[ 3  4  5  6  7]
 [ 8  9 10 11 12]]
Subtraction:
 [[-3 -2 -1  0  1]
 [ 2  3  4  5  6]]
Multiplication:
 [[ 0  3  6  9 12]
 [15 18 21 24 27]]
Division:
 [[0.         0.33333333 0.66666667 1.         1.33333333]
 [1.66666667 2.         2.33333333 2.66666667 3.        ]]
```

```
# Extract the 2nd row and the 3rd column of the reshaped array.
arr2[1, 2]
```

```
np.int64(7)
```

```
[14] # Compute the sum, mean, and standard deviation of all elements in the array.
print("Sum", arr2.sum())
print("Mean", arr2.mean())
print("Standard Deviation", arr2.std())
```

```
Sum 45
Mean 4.5
Standard Deviation 2.8722813232690143
```

### 1.2 Advanced NumPy Operations

```
[15] # Create a 5x5 array of random integers between 1 and 100.
arr_random = np.random.randint(1, 100, size=(5, 5))
arr_random
```

```
array([[16, 32, 71, 99, 17],
       [41, 27, 30, 54, 41],
       [74, 40, 22, 97, 33],
       [37, 54, 19, 46, 78],
       [71, 15, 85, 50, 78]])
```

```
[16] # Find the maximum and minimum values in the array, along with their indices.
print("Max:", arr_random.max())
print("Max Index:", arr_random.argmax())
print("Min:", arr_random.min())
print("Min Index:", arr_random.argmin())
```

```
Max: 99
Max Index: 3
Min: 15
Min Index: 21
```

```
[17] # Extract a sub-array of 3x3 from the center of the array.
sub_arr = arr_random[1:4, 1:4]
sub_arr
```

```
array([[27, 30, 54],
       [40, 22, 97],
       [54, 19, 46]])
```

```
[18] # Perform element-wise operations (addition, subtraction, etc.) between two NumPy arrays of the same shape.
arr_one = np.array([[1, 2, 3], [0, 0, 0]])
arr_two = np.array([[1, 1, 1], [10, 11, 12]])
print("Addition: \n", arr_one + arr_two)
print("Subtraction: \n", arr_one - arr_two)
print("Multiplication: \n", arr_one * arr_two)
print("Division: \n", arr_one / arr_two)
```

```
Addition:
 [[ 2  3  4]
 [10 11 12]]
Subtraction:
 [[  0   1   2]
 [-10 -11 -12]]
Multiplication:
 [[1 2 3]
 [0 0 0]]
Division:
 [[1. 2. 3.]
 [0. 0. 0.]]
```

```
[19] # Use boolean indexing to filter out values greater than 50 in the random array.
arr_random[arr_random > 50]
```

```
array([71, 99, 54, 74, 97, 54, 78, 71, 85, 78])
```

### 1.3 Broadcasting in NumPy

```
# Create a 5x1 column vector (array) and add it to a 5x5 matrix using broadcasting.
arr_col = np.array([[1], [2], [3], [4], [5]])
arr_col + arr_random
```

```
array([[ 17,  33,  72, 100,  18],
       [ 43,  29,  32,  56,  43],
       [ 77,  43,  25, 100,  36],
       [ 41,  58,  23,  50,  82],
       [ 76,  20,  90,  55,  83]])
```

```
[21] # Subtract a 1D array (size 5) from each row of a 5x5 matrix using broadcasting.
arr_row = np.array([1, 2, 3, 4, 5])
arr_random - arr_row
```

```
array([[15, 30, 68, 95, 12],
       [40, 25, 27, 50, 36],
       [73, 38, 19, 93, 28],
       [36, 52, 16, 42, 73],
       [70, 13, 82, 46, 73]])
```

```
[22] # Demonstrate how to use broadcasting to calculate the Euclidean distance between two
     # points in a 2D space (use array-based operations for this).
     import numpy as np
     arr_point_one = np.array([1, 2])
     arr_point_two = np.array([3, 4])
     distance = np.sqrt(np.sum((arr_point_one - arr_point_two) ** 2))
     print(distance)
```

```
2.8284271247461903
```

1.4 Matrix Operations

```
[23] # Create two 3x3 matrices and compute the dot product (matrix multiplication) of the two matrices.
     matrix_one = np.array([[1, 2, 3],
                            [1, 2, 3],
                            [7, 7, 7]])
     matrix_two = np.array([[0, 0, 1],
                            [1, 1, 0],
                            [1, 5, 2]])

     np.dot(matrix_one, matrix_two)
```

```
array([[ 5, 17,  7],
       [ 5, 17,  7],
       [14, 42, 21]])
```

```
     # Calculate the transpose of a matrix and explain the result.
     matrix_three = np.array([[1, 2, 3],
                              [0, 0, 0]])
     matrix_transpose = matrix_three.T
     matrix_transpose

     # The transpose of a matrix swaps its rows and columns. If the original matrix has
     # dimensions m x n, the transposed matrix will have dimensions n x m. So here the
     # original matrix has dimensions 2 x 3, thus the transposed matrix has dimensions 3 x 2
```

```
array([[1, 0],
       [2, 0],
       [3, 0]])
```

```
     # Compute the inverse of a square matrix and explain when an inverse exists.

     square_matrix = np.array([[1, 2],
                               [3, 2]])
     # The determinant = - 4

     def inverse_2x2(matrix):
         a, b = matrix[0]
         c, d = matrix[1]
         determinant = a * d - b * c
         if determinant == 0:
             raise ValueError("Matrix is singular and has no inverse.")
         return (1 / determinant) * np.array([[d, -b], [-c, a]])

     try:
         inverse = inverse_2x2(square_matrix)
         print("Original Matrix:\n", square_matrix)
         print("Inverse:\n", inverse)
     except ValueError as e:
         print(e)

     # Inverse of a square matrix exists if and only if its determinant is non-zero. A square
     # matrix with a non-zero determinant is called non-singular or invertible. If the determinant
     # is zero, the matrix is called singular and does not have an inverse.
```

```
Original Matrix:
 [[1 2]
 [3 2]]
Inverse:
 [[-0.5   0.5 ]
 [ 0.75 -0.25]]
```

# *Part 2 Outputs:*

## Part 2: Pandas Basics and Data Manipulation

### 2.1 Pandas Series and DataFrame Operations

```
[113] # Create a Pandas Series from a list of numbers
      import pandas as pd
      Numbers = [10, 20, 30, 40]
      Series_One = pd.Series(Numbers)
      print(Series_One)
```

```
0    10
1    20
2    30
3    40
dtype: int64
```

```
# Access elements by index.
print(Series_One[0])
print(Series_One[1])
print(Series_One[2])
print(Series_One[3])
```

```
10
20
30
40
```

```
# Perform element-wise arithmetic (e.g., addition of a constant).
print("Addition of a Constant \n")
Series_One + 5
```

```
Addition of a Constant
```

| | 0 |
|---|---|
| 0 | 15 |
| 1 | 25 |
| 2 | 35 |
| 3 | 45 |

**dtype:** int64

```
print("Subtraction of a Constant \n")
Series_One - 10
```

```
Subtraction of a Constant
```

| | 0 |
|---|---|
| 0 | 0 |
| 1 | 10 |
| 2 | 20 |
| 3 | 30 |

**dtype:** int64

```
[117] print("Multiplication of a Constant \n")
      Series_One * 0
```

```
Multiplication of a Constant

   0
0  0
1  0
2  0
3  0

dtype: int64
```

```
print("Division  of a Constant \n")
Series_One / 10
```

```
Division  of a Constant

     0
0  1.0
1  2.0
2  3.0
3  4.0

dtype: float64
```

```
[119] # Filter out values greater than 50.
      print(Series_One[Series_One > 50])
```

```
Series([], dtype: int64)
```

```
[120] # Create the dictionary
      dictionary_data = {'Name': ['Alice', 'Lili', 'Mira', 'Jouri'],
                         'Age': [20, 22, 15, 18],
                         'Score': [85.5, 78.0, 95.3, 88.7]}

      # Create the Pandas DataFrame
      df = pd.DataFrame(dictionary_data)
      print(df)
```

```
    Name  Age  Score
0  Alice   20   85.5
1   Lili   22   78.0
2   Mira   15   95.3
3  Jouri   18   88.7
```

```
      # Access specific rows and columns using .loc and .iloc.

      # Access row with index 1 and columns 'Name' and 'Age'
      print(df.loc[1, ['Name', 'Age']])
```

```
Name    Lili
Age       22
Name: 1, dtype: object
```

```
[122] # Access rows with index 2 and 3 and all columns
      print(df.loc[[2, 3], :])
```

```
    Name  Age  Score
2   Mira   15   95.3
3  Jouri   18   88.7
```

```
[123] # Access row with integer index 0 and columns at integer indices 0 and 2
      print(df.iloc[0, [0, 2]])
```

```
Name     Alice
Score     85.5
Name: 0, dtype: object
```

```
[124] # Access rows with integer indices 1 and 3 and all columns
      print(df.iloc[[1, 3], :])
```

```
    Name  Age  Score
1   Lili   22   78.0
3  Jouri   18   88.7
```

```
[125] # Filter rows based on a condition (e.g., select all students older than 18).

      print("Students older than 18 years: \n", df[df['Age'] > 18])
      print("---------------")
      print("Student with +A grade: \n", df[df['Score'] >= 95])
```

```
Students older than 18 years:
     Name  Age  Score
0  Alice   20   85.5
1   Lili   22   78.0
---------------
Student with +A grade:
    Name  Age  Score
2  Mira   15   95.3
```

```
      # Sort the DataFrame based on 'Age' in ascending order and 'Score' in descending order.
      sorted_df = df.sort_values(by=['Age', 'Score'], ascending=[True, False])
      print(sorted_df)
```

```
    Name  Age  Score
2   Mira   15   95.3
3  Jouri   18   88.7
0  Alice   20   85.5
1   Lili   22   78.0
```

## 2.2 Data Cleaning and Transformation

```
      # Load the provided CSV file into a Pandas DataFrame.
      df_students = pd.read_csv('students_data.csv')
      df_students.head()
```

| | Student_ID | Name | Age | Score |
|---|---|---|---|---|
| 0 | 0 | Kiana Lor | 22 | 2.825 |
| 1 | 1 | Joshua Lonaker | 22 | 3.675 |
| 2 | 2 | Dakota Blanco | 22 | 3.975 |
| 3 | 3 | Natasha Yarusso | 20 | 3.575 |
| 4 | 4 | Brooke Cazares | 21 | 2.675 |

Next steps: Generate code with df_students | View recommended plots | New interactive sheet

## Data Cleaning

```
      # Handle missing values by either filling them with the mean of the column or dropping them.
      print("Missing Values \n", df_students.isnull().sum())
```

```
Missing Values
 Student_ID    0
Name           0
Age            0
Score          0
dtype: int64
```

There are no missing values

```
[129] # If there were some missing values and we want to drop them we would wrote
      # df_studens.dropna(inplace=True)
```

```
[130] # Check for duplicates and remove them if any.
      print("Number of duplicate rows:", df_students.duplicated().sum())
```

```
Number of duplicate rows: 0
```

There are no duplicated rows

```
[131] # If there were some duplicated rows and we want to remove them we would wrote
      # df_students.drop_duplicates(inplace=True)
```

```
# Convert a column 'Age' into a categorical variable with bins: 'Young' (0-20), 'Middle-aged' (21-40), 'Old' (41+).

if 'Age' in df_students.columns:
    bins = [0, 20, 40, float('inf')]
    labels = ['Young', 'Middle-aged', 'Old']
    df_students['Age Group'] = pd.cut(df_students['Age'], bins=bins, labels=labels, right=False)
    print(df_students.head())
else:
    print("Column 'Age' not found in the DataFrame.")
```

```
   Student_ID            Name  Age  Score    Age Group
0           0       Kiana Lor   22  2.825  Middle-aged
1           1   Joshua Lonaker   22  3.675  Middle-aged
2           2    Dakota Blanco   22  3.975  Middle-aged
3           3  Natasha Yarusso   20  3.575  Middle-aged
4           4   Brooke Cazares   21  2.675  Middle-aged
```

Since in the students_csv dataset, the score is out of 5 (gpa). I will translate the 60 and 50 in the following instructions to be out of 5.

```
[133] # Add a new column 'Passed' that contains 'Yes' if the student's score is greater than or equal to 60, otherwise 'No'.

df_students['Passed'] = df_students['Score'].apply(lambda x: 'Yes' if x >= 2.75 else 'No')
print(df_students.head())
```

```
   Student_ID            Name  Age  Score    Age Group Passed
0           0       Kiana Lor   22  2.825  Middle-aged    Yes
1           1   Joshua Lonaker   22  3.675  Middle-aged    Yes
2           2    Dakota Blanco   22  3.975  Middle-aged    Yes
3           3  Natasha Yarusso   20  3.575  Middle-aged    Yes
4           4   Brooke Cazares   21  2.675  Middle-aged     No
```

## 2.3 Grouping and Aggregation

```
# Adde the 'Passed' column
df_students['Passed'] = df_students['Score'].apply(lambda score: 'Yes' if score >= 2.5 else 'No')


# Group the data by the 'Passed' column and compute the mean, median, and standard deviation of the 'Age' and 'Score' columns for each group.
grouped_data = df_students.groupby('Passed').agg(
    {
        'Age': ['mean', 'median', 'std'],
        'Score': ['mean', 'median', 'std']
    }
)
grouped_data.head()
```

| | Age | | | Score | | |
|---|---|---|---|---|---|---|
| | mean | median | std | mean | median | std |
| **Passed** | | | | | | |
| **No** | 23.000000 | 23.0 | NaN | 2.300000 | 2.300 | NaN |
| **Yes** | 21.960784 | 22.0 | 1.248644 | 3.661193 | 3.725 | 0.328957 |

```
[136] # Find the count of students in each 'Age' category (young, middle-aged, old).
      df_students['Age Group'].value_counts()
```

| Age Group | count |
|---|---|
| Middle-aged | 303 |
| Young | 4 |
| Old | 0 |

dtype: int64

```
# Use the pivot_table() method to compute the average score by age category.
pivot_average_score = pd.pivot_table(df_students, values='Score',
                                     index='Age Group', aggfunc='mean')

print("Average score by Age category (using pivot_table): \n", pivot_average_score )
```

```
Average score by Age category (using pivot_table):
                Score
Age Group
Young        3.850000
Middle-aged  3.654208
<ipython-input-137-b0704ea206ca>:2: FutureWarning: The default value of observed=False is deprecated and will change to observed=Tr
  pivot_average_score = pd.pivot_table(df_students, values='Score',
```

# *Part 3 Outputs:*

## Part 3: Advanced Pandas and Real-World Application

### 3.1 Data Exploration and Analysis (Real-world Dataset)

```
[27] # Load the dataset into a Pandas DataFrame
     import pandas as pd
     titanic_df = pd.read_csv('titanic.csv')
```

```
[28] # Display the first 10 rows of the dataset.
     titanic_df.head(10)
```

| | PassengerId | Survived | Pclass | Name | Sex | Age | SibSp | Parch | Ticket | Fare | Cabin | Embarked |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 0 | 3 | Braund, Mr. Owen Harris | male | 22.0 | 1 | 0 | A/5 21171 | 7.2500 | NaN | S |
| 1 | 2 | 1 | 1 | Cumings, Mrs. John Bradley (Florence Briggs Th... | female | 38.0 | 1 | 0 | PC 17599 | 71.2833 | C85 | C |
| 2 | 3 | 1 | 3 | Heikkinen, Miss. Laina | female | 26.0 | 0 | 0 | STON/O2. 3101282 | 7.9250 | NaN | S |
| 3 | 4 | 1 | 1 | Futrelle, Mrs. Jacques Heath (Lily May Peel) | female | 35.0 | 1 | 0 | 113803 | 53.1000 | C123 | S |
| 4 | 5 | 0 | 3 | Allen, Mr. William Henry | male | 35.0 | 0 | 0 | 373450 | 8.0500 | NaN | S |
| 5 | 6 | 0 | 3 | Moran, Mr. James | male | NaN | 0 | 0 | 330877 | 8.4583 | NaN | Q |
| 6 | 7 | 0 | 1 | McCarthy, Mr. Timothy J | male | 54.0 | 0 | 0 | 17463 | 51.8625 | E46 | S |
| 7 | 8 | 0 | 3 | Palsson, Master. Gosta Leonard | male | 2.0 | 3 | 1 | 349909 | 21.0750 | NaN | S |
| 8 | 9 | 1 | 3 | Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg) | female | 27.0 | 0 | 2 | 347742 | 11.1333 | NaN | S |
| 9 | 10 | 1 | 2 | Nasser, Mrs. Nicholas (Adele Achem) | female | 14.0 | 1 | 0 | 237736 | 30.0708 | NaN | C |

Next steps: Generate code with titanic_df | View recommended plots | New interactive sheet

```
# Check the summary statistics (e.g., mean, standard deviation, etc.) for numeric columns.
titanic_df.describe()
```

| | PassengerId | Survived | Pclass | Age | SibSp | Parch | Fare |
|---|---|---|---|---|---|---|---|
| count | 891.000000 | 891.000000 | 891.000000 | 714.000000 | 891.000000 | 891.000000 | 891.000000 |
| mean | 446.000000 | 0.383838 | 2.308642 | 29.699118 | 0.523008 | 0.381594 | 32.204208 |
| std | 257.353842 | 0.486592 | 0.836071 | 14.526497 | 1.102743 | 0.806057 | 49.693429 |
| min | 1.000000 | 0.000000 | 1.000000 | 0.420000 | 0.000000 | 0.000000 | 0.000000 |
| 25% | 223.500000 | 0.000000 | 2.000000 | 20.125000 | 0.000000 | 0.000000 | 7.910400 |
| 50% | 446.000000 | 0.000000 | 3.000000 | 28.000000 | 0.000000 | 0.000000 | 14.454200 |
| 75% | 668.500000 | 1.000000 | 3.000000 | 38.000000 | 1.000000 | 0.000000 | 31.000000 |
| max | 891.000000 | 1.000000 | 3.000000 | 80.000000 | 8.000000 | 6.000000 | 512.329200 |

```
# Handle missing data by either filling or removing it (depending on the type of data and the column).

# Get the count of missing values per column
titanic_df.isnull().sum()
```

| | 0 |
|---|---|
| PassengerId | 0 |
| Survived | 0 |
| Pclass | 0 |
| Name | 0 |
| Sex | 0 |
| Age | 177 |
| SibSp | 0 |
| Parch | 0 |
| Ticket | 0 |
| Fare | 0 |
| Cabin | 687 |
| Embarked | 2 |

dtype: int64

```
titanic_df.dtypes
```

| | 0 |
|---|---|
| PassengerId | int64 |
| Survived | int64 |
| Pclass | int64 |
| Name | object |
| Sex | object |
| Age | float64 |
| SibSp | int64 |
| Parch | int64 |
| Ticket | object |
| Fare | float64 |
| Cabin | object |
| Embarked | object |

dtype: object

For Age and Embarked: I filled the missing values with the mode.

For Cabin: Given the high number of missing values, I dropped the Cabin column.

```
[32] # Fill missing 'Embarked' values with the mode (most frequent value)
mode_embarked = titanic_df['Embarked'].mode()[0]
titanic_df['Embarked'].fillna(mode_embarked, inplace=True)
print(f"Filled missing 'Embarked' values with: {mode_embarked}")
```

```
Filled missing 'Embarked' values with: S
<ipython-input-32-0801366c8d29>:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  titanic_df['Embarked'].fillna(mode_embarked, inplace=True)
```

```
# Fill missing 'Age' values with the median
median_age = titanic_df['Age'].median()
titanic_df['Age'].fillna(median_age, inplace=True)
print(f"Filled missing 'Age' values with: {median_age}")
```

```
Filled missing 'Age' values with: 28.0
<ipython-input-33-d95c7c5f3e0c>:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.


  titanic_df['Age'].fillna(median_age, inplace=True)
```

```
[34] # Drop the 'Cabin' column
titanic_df.drop('Cabin', axis=1, inplace=True)
print("Dropped the 'Cabin' column")
```

```
Dropped the 'Cabin' column
```

```
# Verify the missing values after dropping 'Cabin' and filling 'Embarked' and 'Age'
print("Missing values after handling:")
print(titanic_df.isnull().sum())
```

```
Missing values after handling:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Embarked       0
dtype: int64
```

Convert categorical columns into numeric ones using encoding techniques such as pd.get_dummies().

```
[36] # Identify categorical columns
categorical_cols = titanic_df.select_dtypes(include=['object']).columns

print("Categorical columns before encoding:", categorical_cols)
```

```
Categorical columns before encoding: Index(['Name', 'Sex', 'Ticket', 'Embarked'], dtype='object')
```

```
# Apply pd.get_dummies() for one-hot encoding
titanic_df = pd.get_dummies(titanic_df, columns=categorical_cols, drop_first=True)

print("\nDataFrame after one-hot encoding:")
print(titanic_df.head())
```

```
DataFrame after one-hot encoding:
   PassengerId  Survived  Pclass   Age  SibSp  Parch     Fare  \
0            1         0       3  22.0      1      0   7.2500
1            2         1       1  38.0      1      0  71.2833
2            3         1       3  26.0      0      0   7.9250
3            4         1       1  35.0      1      0  53.1000
4            5         0       3  35.0      0      0   8.0500

   Name_Abbott, Mr. Rossmore Edward  Name_Abbott, Mrs. Stanton (Rosa Hunt)  \
0                             False                                  False
1                             False                                  False
2                             False                                  False
3                             False                                  False
4                             False                                  False

   Name_Abelson, Mr. Samuel  ...  Ticket_W./C. 14258  Ticket_W./C. 14263  \
0                     False  ...               False               False
1                     False  ...               False               False
2                     False  ...               False               False
3                     False  ...               False               False
4                     False  ...               False               False

   Ticket_W./C. 6607  Ticket_W./C. 6608  Ticket_W./C. 6609  \
0              False              False              False
1              False              False              False
2              False              False              False
3              False              False              False
4              False              False              False

   Ticket_W.E.P. 5734  Ticket_W/C 14208  Ticket_WE/P 5735  Embarked_Q  \
0               False             False             False       False
1               False             False             False       False
2               False             False             False       False
3               False             False             False       False
4               False             False             False       False

   Embarked_S
0        True
1       False
2        True
3        True
4        True

[5 rows x 1580 columns]
```

```
[70] print("Shape of DataFrame after encoding:", titanic_df.shape)
```

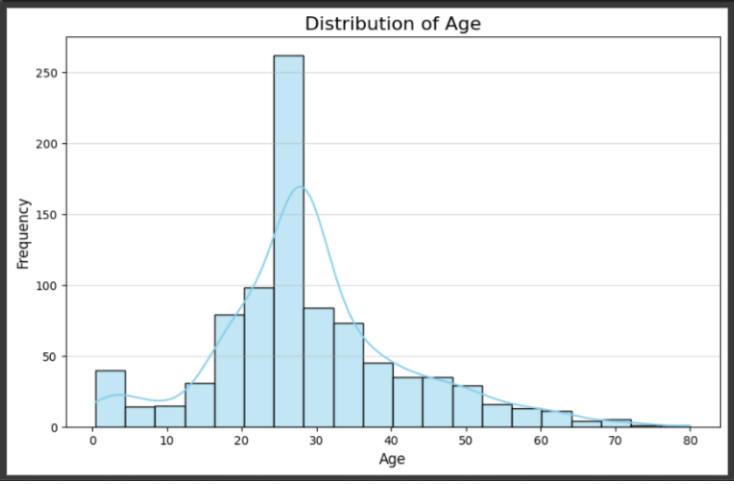
```
Shape of DataFrame after encoding: (891, 1580)
```

```
print("Data types after encoding:")
print(titanic_df.dtypes)
```

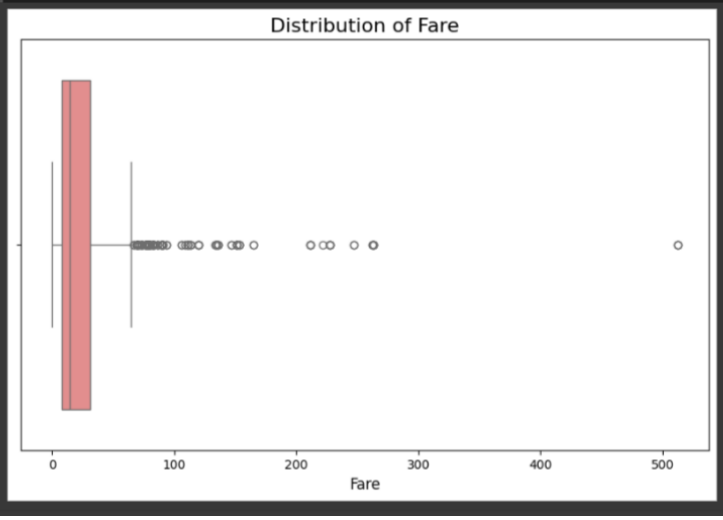
```
Data types after encoding:
PassengerId             int64
Survived                int64
Pclass                  int64
Age                   float64
SibSp                   int64
                       ...
Ticket_W.E.P. 5734       bool
Ticket_W/C 14208         bool
Ticket_WE/P 5735         bool
Embarked_Q               bool
Embarked_S               bool
Length: 1580, dtype: object
```

## 3.2 Data Visualization with Pandas and Matplotlib

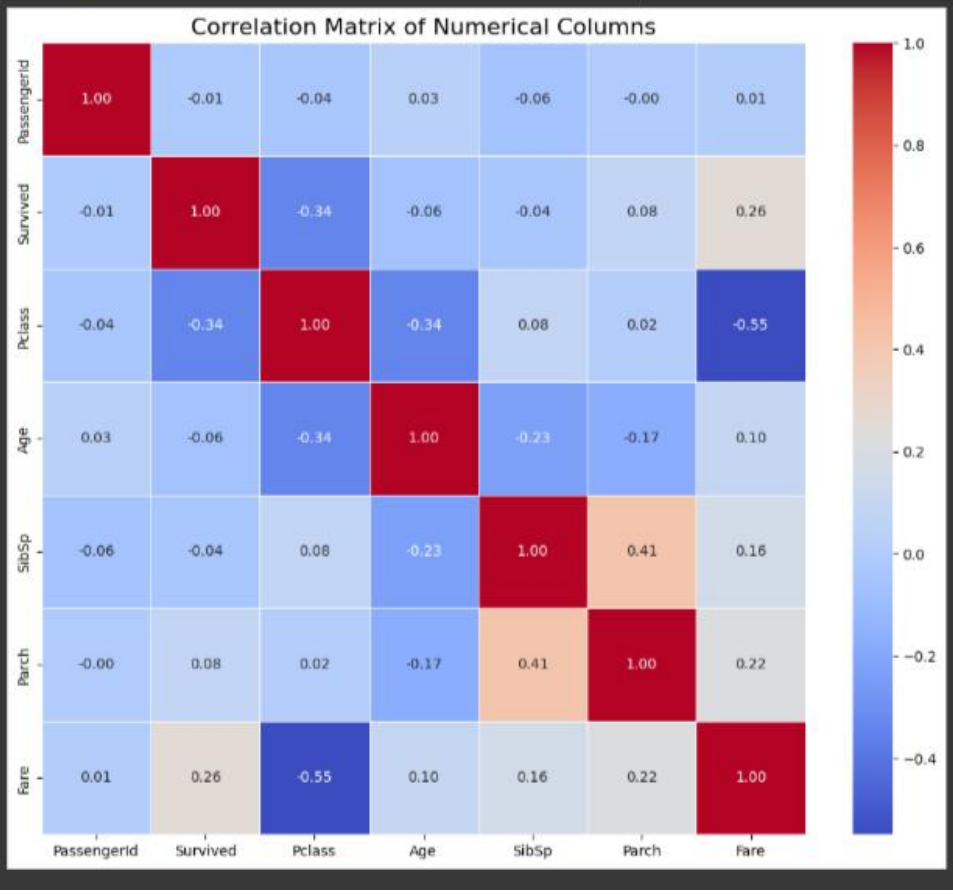
```
# Use Pandas to plot:
# A histogram of a numeric column.

import matplotlib.pyplot as plt
import seaborn as sns
# Histogram of Age
plt.figure(figsize=(10, 6))
sns.histplot(titanic_df['Age'], bins=20, kde=True, color='skyblue')
plt.title('Distribution of Age', fontsize=16)
plt.xlabel('Age', fontsize=12)
plt.ylabel('Frequency', fontsize=12)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.grid(axis='y', alpha=0.5)
plt.show()
```



```python
# A box plot to visualize the distribution of a column.

# Box Plot of Fare
plt.figure(figsize=(10, 6))
sns.boxplot(x=titanic_df['Fare'], color='lightcoral')
plt.title('Distribution of Fare', fontsize=16)
plt.xlabel('Fare', fontsize=12)
plt.xticks(fontsize=10)
plt.show()
```



```python
# A correlation matrix to explore the relationships between numerical columns.

# Correlation Matrix of Numerical Columns
numerical_cols = titanic_df.select_dtypes(include=['int64', 'float64']).columns
correlation_matrix = titanic_df[numerical_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5, annot_kws={"size": 10})
plt.title('Correlation Matrix of Numerical Columns', fontsize=16)
plt.xticks(fontsize=10)
plt.yticks(fontsize=10)
plt.show()
```



## 3.3 Time-Series Data

The Electricity Transformer Dataset (ETDataset) is a time-series dataset that I got from GitHub. It spans from 2016/07 to 2018/07 and comprises three variations: ETT-small (data from 2 transformers), ETT-large (data from 39 transformers), and ETT-full (data from 69 transformer stations). These datasets include measurements of load and oil temperature. ETT-full also includes location, climate, and demand data.

```
[50] # Load a time-series dataset

     EET_df = pd.read_csv('ETTm1.csv')
     EET_df.head()
```

| | date | HUFL | HULL | MUFL | MULL | LUFL | LULL | OT |
|---|---|---|---|---|---|---|---|---|
| 0 | 2016-07-01 00:00:00 | 5.827 | 2.009 | 1.599 | 0.462 | 4.203 | 1.340 | 30.531000 |
| 1 | 2016-07-01 00:15:00 | 5.760 | 2.076 | 1.492 | 0.426 | 4.264 | 1.401 | 30.459999 |
| 2 | 2016-07-01 00:30:00 | 5.760 | 1.942 | 1.492 | 0.391 | 4.234 | 1.310 | 30.038000 |
| 3 | 2016-07-01 00:45:00 | 5.760 | 1.942 | 1.492 | 0.426 | 4.234 | 1.310 | 27.013000 |
| 4 | 2016-07-01 01:00:00 | 5.693 | 2.076 | 1.492 | 0.426 | 4.142 | 1.371 | 27.787001 |

Next steps: Generate code with EET_df | View recommended plots | New interactive sheet

```
EET_df['date']
```

| | date |
|---|---|
| 0 | 2016-07-01 00:00:00 |
| 1 | 2016-07-01 00:15:00 |
| 2 | 2016-07-01 00:30:00 |
| 3 | 2016-07-01 00:45:00 |
| 4 | 2016-07-01 01:00:00 |
| ... | ... |

| | | |
|---|---|---|
| | ... | ... |
| | **69675** | 2018-06-26 18:45:00 |
| | **69676** | 2018-06-26 19:00:00 |
| | **69677** | 2018-06-26 19:15:00 |
| | **69678** | 2018-06-26 19:30:00 |
| | **69679** | 2018-06-26 19:45:00 |

69680 rows × 1 columns

**dtype:** object

```
[51] EET_df['date'].dtypes
```

```
dtype('O')
```

```
[54] # Convert the 'Date' column to a datetime object.
     EET_df['date'] = pd.to_datetime(EET_df['date'])
```

```
[57] EET_df.dtypes
```

| | 0 |
|---|---|
| **date** | datetime64[ns] |
| **HUFL** | float64 |
| **HULL** | float64 |
| **MUFL** | float64 |
| **MULL** | float64 |
| **LUFL** | float64 |
| **LULL** | float64 |
| **OT** | float64 |

| | |
|---|---|
| LULL | float64 |
| OT | float64 |

**dtype:** object

```
[58] EET_df['date'].dtypes
```

```
dtype('<M8[ns]')
```

```
[60] EET_df.sample(3)
```

| | date | HUFL | HULL | MUFL | MULL | LUFL | LULL | OT |
|---|---|---|---|---|---|---|---|---|
| 53396 | 2018-01-08 05:00:00 | 13.731 | 3.751 | 11.087 | 2.559 | 2.224 | 0.457 | 0.633 |
| 51511 | 2017-12-19 13:45:00 | -7.569 | 2.277 | -10.909 | 0.959 | 2.833 | 1.127 | 5.276 |
| 38799 | 2017-08-09 03:45:00 | 10.248 | 0.402 | 7.356 | -0.569 | 2.467 | 0.853 | 15.054 |

```
# Set the 'Date' column as the index of the DataFrame.

EET_df.set_index('date', inplace=True)
```

```
[62] EET_df.index
```

```
DatetimeIndex(['2016-07-01 00:00:00', '2016-07-01 00:15:00',
               '2016-07-01 00:30:00', '2016-07-01 00:45:00',
               '2016-07-01 01:00:00', '2016-07-01 01:15:00',
               '2016-07-01 01:30:00', '2016-07-01 01:45:00',
               '2016-07-01 02:00:00', '2016-07-01 02:15:00',
               ...
               '2018-06-26 17:30:00', '2018-06-26 17:45:00',
               '2018-06-26 18:00:00', '2018-06-26 18:15:00',
               '2018-06-26 18:30:00', '2018-06-26 18:45:00',
               '2018-06-26 19:00:00', '2018-06-26 19:15:00',
               '2018-06-26 19:30:00', '2018-06-26 19:45:00'],
              dtype='datetime64[ns]', name='date', length=69680, freq=None)
```

```
[63] EET_df.tail()
```

| date | HUFL | HULL | MUFL | MULL | LUFL | LULL | OT |
|---|---|---|---|---|---|---|---|
| 2018-06-26 18:45:00 | 9.310 | 3.550 | 5.437 | 1.670 | 3.868 | 1.462 | 9.567 |
| 2018-06-26 19:00:00 | 10.114 | 3.550 | 6.183 | 1.564 | 3.716 | 1.462 | 9.567 |
| 2018-06-26 19:15:00 | 10.784 | 3.349 | 7.000 | 1.635 | 3.746 | 1.432 | 9.426 |
| 2018-06-26 19:30:00 | 11.655 | 3.617 | 7.533 | 1.706 | 4.173 | 1.523 | 9.426 |
| 2018-06-26 19:45:00 | 12.994 | 3.818 | 8.244 | 1.777 | 4.721 | 1.523 | 9.778 |

```
[64] # Resample to monthly frequency and calculate the average
monthly_avg = EET_df.resample('M').mean()
```

```
<ipython-input-64-ce870fe8579a>:2: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.
  monthly_avg = EET_df.resample('M').mean()
```

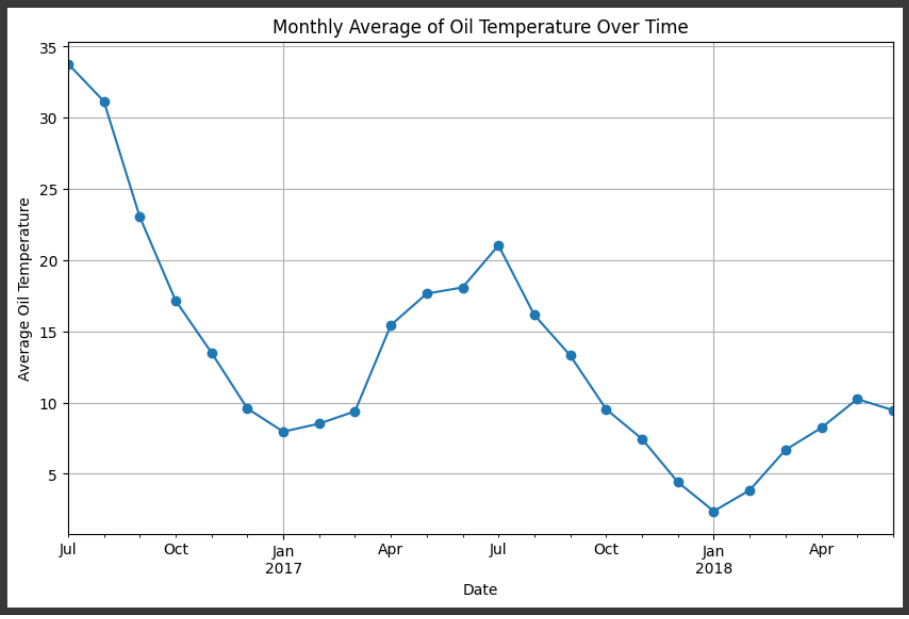
```
print(monthly_avg)
```

```
                HUFL      HULL      MUFL      MULL      LUFL      LULL  \
date
2016-07-31  11.958810  3.381800  8.187963  1.321868  3.668116  1.400037
2016-08-31  13.012467  4.927991  9.192595  2.944231  3.756678  1.215012
2016-09-30   9.044914  4.103064  6.465602  2.600920  2.484533  1.002549
2016-10-31   8.711088  2.899241  6.252790  2.116707  2.370440  0.346446
2016-11-30   9.259652  2.894568  6.727288  2.402553  2.456675  0.011149
2016-12-31   9.172616  1.111888  6.583028 -0.174139  2.540894  0.832348
2017-01-31   9.020981 -0.268342  6.121396 -1.583794  2.827293  0.735756
2017-02-28   7.212612 -0.272016  4.201330 -1.531462  2.949728  0.777681
2017-03-31   6.602462  0.524543  3.834800 -0.885005  2.698084  0.965659
2017-04-30   3.960243  0.859835  1.326834 -0.612551  2.554742  1.047939
2017-05-31   2.426288  2.155474 -0.272483  0.786385  2.633674  1.064817
2017-06-30   4.932522  1.959487  2.080975  1.772761  2.764938 -0.143864
2017-07-31   7.471564  1.493721  4.093075  0.682258  4.878149  0.292909
2017-08-31   7.405061  2.423494  3.703144  0.685475  3.631290  1.183887
2017-09-30   5.295251  0.065061  2.766672 -1.125054  2.442870  0.716282
2017-10-31   5.267415  2.531314  2.841367  1.328011  2.335074  0.775651
```

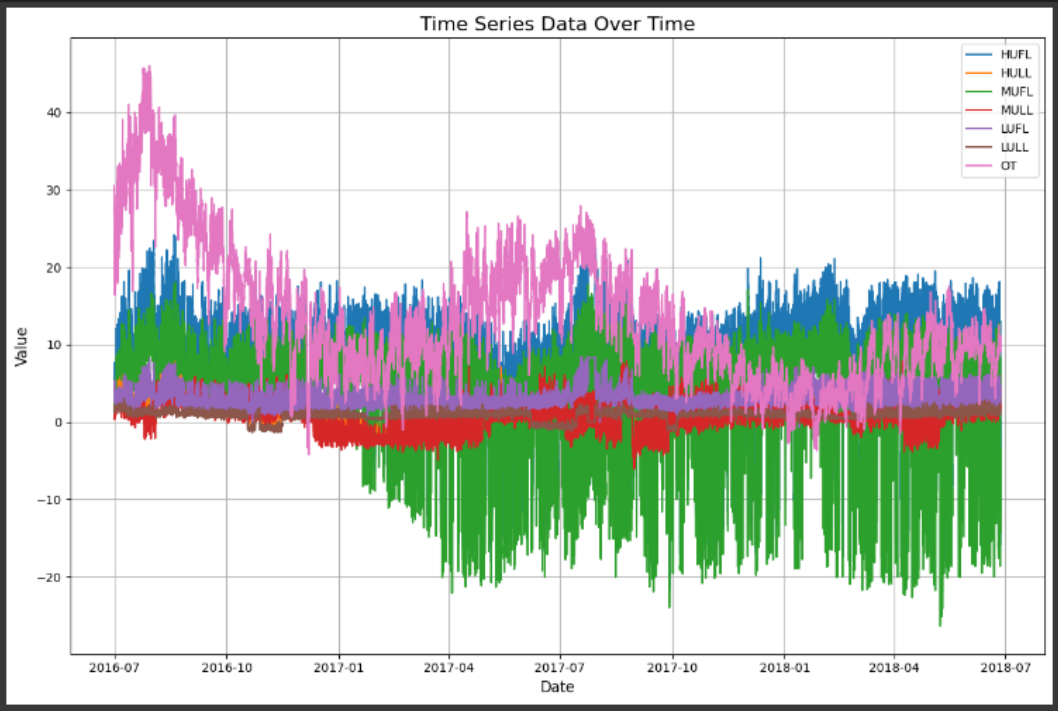
```
2017-12-31   8.282917  3.226711  4.824364  1.630870  3.394889  1.058570
2018-01-31  10.837535  3.130003  7.036321  1.609055  3.718738  0.866258
2018-02-28   7.687491  1.879710  4.310750  0.722648  3.314901  0.608982
2018-03-31   6.215693  3.010544  2.907852  1.560415  3.231289  0.933429
2018-04-30   5.620571  1.890914  2.154146  0.259650  3.399455  1.092676
2018-05-31   6.336279  3.046561  2.769660  1.281631  3.473296  1.231069
2018-06-30   5.863612  4.333677  2.072035  2.333244  3.686938  1.428610

                  OT
date
2016-07-31  33.781010
2016-08-31  31.144733
2016-09-30  23.037765
2016-10-31  17.155029
2016-11-30  13.497975
2016-12-31   9.577848
2017-01-31   7.952813
2017-02-28   8.512802
2017-03-31   9.380204
2017-04-30  15.431626
2017-05-31  17.643219
2017-06-30  18.077689
2017-07-31  21.033239
2017-08-31  16.138329
2017-09-30  13.301285
2017-10-31   9.530860
2017-11-30   7.441660
2017-12-31   4.410764
2018-01-31   2.371807
2018-02-28   3.840313
2018-03-31   6.673227
2018-04-30   8.222539
2018-05-31  10.238203
2018-06-30   9.460354
```

```
[66] # Visualize the resampled data (e.g., for 'OT' - Oil Temperature)
     plt.figure(figsize=(10, 6))
     monthly_avg['OT'].plot(marker='o', linestyle='-')
     plt.title('Monthly Average of Oil Temperature Over Time')
     plt.xlabel('Date')
     plt.ylabel('Average Oil Temperature')
     plt.grid(True)
```

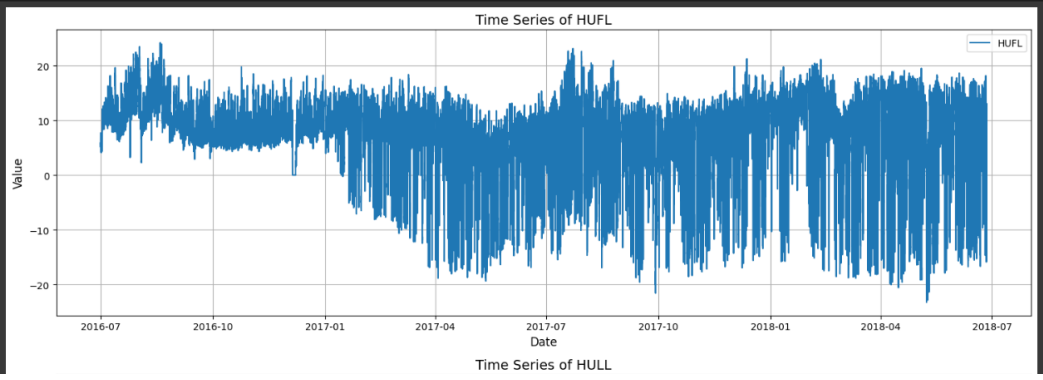
```
# Visualize the resampled data (e.g., for 'OT' - Oil Temperature)
plt.figure(figsize=(10, 6))
monthly_avg['OT'].plot(marker='o', linestyle='-')
plt.title('Monthly Average of Oil Temperature Over Time')
plt.xlabel('Date')
plt.ylabel('Average Oil Temperature')
plt.grid(True)
```

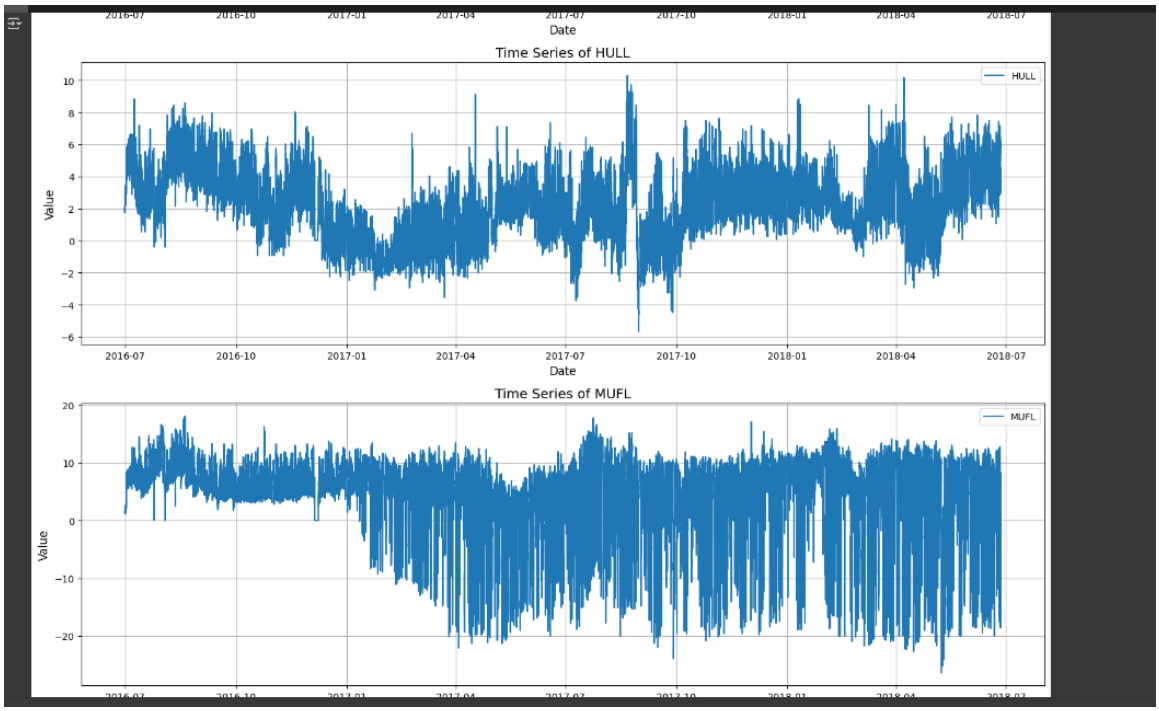


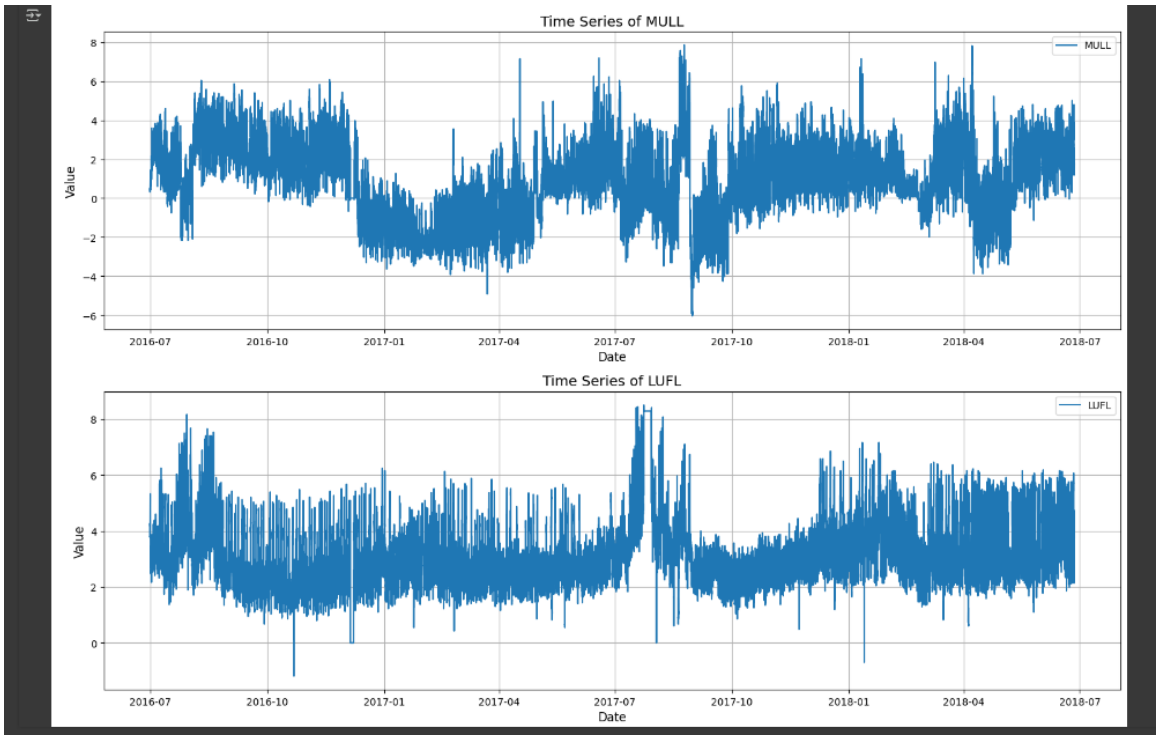
```python
# Plot the time series data over time.

# Plot all the time series data over time
plt.figure(figsize=(12, 8))
for column in EET_df.columns:
    plt.plot(EET_df.index, EET_df[column], label=column)

plt.title('Time Series Data Over Time', fontsize=16)
plt.xlabel('Date', fontsize=12)
plt.ylabel('Value', fontsize=12)
plt.legend(fontsize=10)
plt.grid(True)
plt.tight_layout()
plt.show()
```

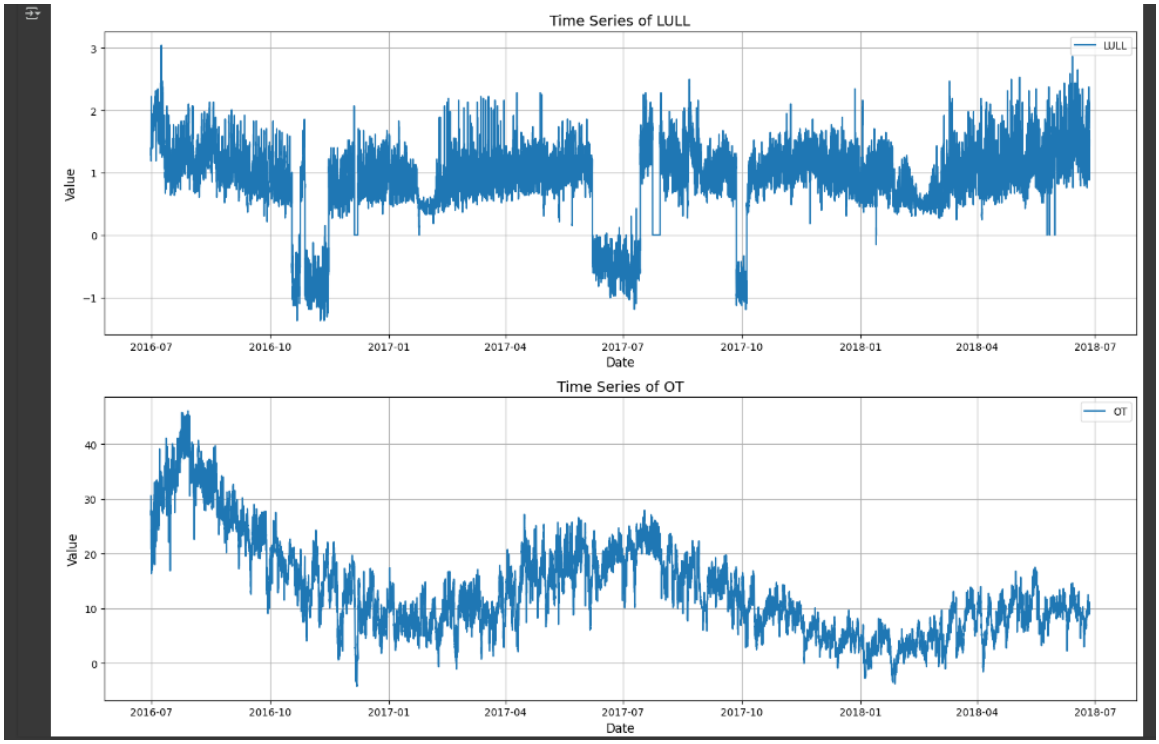



```python
# Plot each time series in a separate subplot:
num_cols = len(EET_df.columns)
plt.figure(figsize=(15, 5 * num_cols))
for i, column in enumerate(EET_df.columns):
    plt.subplot(num_cols, 1, i + 1)
    plt.plot(EET_df.index, EET_df[column], label=column)
    plt.title(f'Time Series of {column}', fontsize=14)
    plt.xlabel('Date', fontsize=12)
    plt.ylabel('Value', fontsize=12)
    plt.legend(fontsize=10)
    plt.grid(True)
plt.tight_layout()
plt.show()
```




Date
Time Series of HULL
HULL
Value
Date
Time Series of MUFL
MUFL
Value


Time Series of MULL
MULL
Value
Date
Time Series of LUFL
LUFL
Value
Date


Time Series of LULL
LULL
Value
Date
Time Series of OT
OT
Value
Date
2016-07
2016-10
2017-01
2017-04
2017-07
2017-10
2018-01
2018-04
2018-07